



UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de computadores

Proyecto 1 - Diseño Lógico y FPGA

Grupo 4:

Estudiante 1: Christian Georgi

Estudiante 2: Diego Quinteros

Estudiante 3: Felipe Riveros

Profesor de Cátedra: Jorge Gómez Mir

9 de Septiembre de 2026

Índice

1. Introducción	3
2. Diseño general y arquitectura	5
2.1. Diseño General	5
2.2. Arquitectura	7
3. Operaciones de la calculadora	9
4. Diseño lógico de la calculadora	10
4.1. Sumador completo de 1 bit	10
4.1.1. Tabla de verdad	10
4.1.2. Mapas de Karnaugh	10
4.1.3. Expresiones booleanas	11
4.2. Sumador de 4 bits	12
4.3. Restador mediante complemento a 2	12
4.3.1. Tabla de verdad	12
4.3.2. Mapa de Karnaugh	13
4.3.3. Expresiones booleanas	14
4.4. Multiplexor 2:1	15
4.4.1. Tabla de verdad	15
4.4.2. Mapas de Karnaugh	15
4.4.3. Expresiones booleanas	15
4.5. Selector de operación	16
4.5.1. Tabla de verdad	16
4.5.2. Mapas de Karnaugh	16
4.5.3. Expresiones booleanas	18
4.6. Shift Left	19
4.6.1. Tabla de verdad	19
4.6.2. Mapas de Karnaugh	19
4.6.3. Expresiones booleanas	20

4.7. Shift Right	21
4.7.1. Tabla de verdad	21
4.7.2. Mapas de Karnaugh	21
4.7.3. Expresiones booleanas	22
4.8. Registro y señal de ejecución	23
4.8.1. Tabla de funcionamiento	23
4.8.2. Funcionamiento	23
4.8.3. Implementación	23
5. Implementación mediante compuertas	25
5.1. Implementación del sumador	25
5.2. Implementación del restador	25
5.3. Implementación del Shift Left	26
5.4. Implementación del Shift Right	26
5.5. Implementación del selector del segundo operando	26
5.6. Implementación del selector de operación	26
5.7. Implementación del registro de resultado	27
5.8. Integración de los módulos	27
6. Declaración de uso de IA	28
7. Bibliografía	29

1. Introducción

El proyecto se enfoca en diseñar e implementar una calculadora de 4 bits utilizando Verilog, para posteriormente implementarla en una FPGA Lattice iCE40 HX1K. El trabajo se desarrolló utilizando únicamente diseño lógico y compuertas lógicas para realizar las distintas operaciones de la calculadora. Su funcionamiento fue verificado tanto mediante simulación como a través de pruebas directas en la FPGA.

Este proyecto permite aplicar operaciones aritméticas y lógicas mediante circuitos digitales funcionales, integrando los distintos elementos necesarios para conformar un sistema digital completo, tales como la selección de operandos y el almacenamiento de resultados. Los valores de 4 bits son representados mediante complemento a dos, permitiendo trabajar tanto con números positivos como negativos.

Se plantea que, mediante la correcta implementación y conexión de los distintos bloques lógicos, es posible construir una calculadora de 4 bits capaz de realizar las operaciones requeridas, almacenar el resultado obtenido y reutilizarlo como segundo operando en una operación posterior.

Objetivo general

Diseñar e implementar una calculadora digital de 4 bits utilizando Verilog y únicamente compuertas lógicas, para posteriormente verificar su funcionamiento mediante simulaciones e implementarla en una FPGA.

Objetivos específicos

- Diseñar las operaciones aritméticas, lógicas y de desplazamiento de bits mediante tablas de verdad, mapas de Karnaugh y expresiones booleanas.
- Implementar la lógica combinacional de la calculadora utilizando únicamente compuertas lógicas y su correspondiente descripción en Verilog.
- Implementar un sistema que permita seleccionar el segundo operando entre un valor externo y el resultado de la operación anterior, almacenando este último mediante un registro de 4 bits.

Este informe se estructurará de la siguiente manera: en primer lugar, se presentará el diseño general y la arquitectura de la calculadora. Posteriormente, se describirán las operaciones requeridas y el diseño lógico de cada uno de sus componentes (utilizando tablas de verdad, mapas de Karnaugh y expresiones booleanas). Luego, se expondrá la implementación mediante compuertas lógicas.

2. Diseño general y arquitectura

2.1. Diseño General

Para el diseño de la calculadora se separaron las distintas funciones en bloques independientes, con el objetivo de facilitar tanto su implementación como la comprensión de su funcionamiento. Los principales bloques corresponden al sumador, restador, restador inverso, desplazamiento hacia la izquierda, desplazamiento hacia la derecha, selector del segundo operando, selector de operación y registro de resultado.

La calculadora trabaja con operandos y resultados de 4 bits. Los valores pueden ser interpretados mediante complemento a dos, permitiendo representar tanto números positivos como negativos dentro del rango disponible para esta cantidad de bits.

Para realizar una operación, la calculadora recibe un primer operando denominado `op1` y un segundo operando. Este último puede provenir de una entrada externa, denominada `op2_ext`, o del resultado almacenado de la operación anterior. La señal `sel_op2` determina cuál de estas dos fuentes será utilizada como segundo operando, permitiendo encadenar distintas operaciones sin tener que ingresar nuevamente el resultado anterior.

Una vez definidos ambos operandos, estos son enviados a los distintos bloques encargados de realizar las operaciones disponibles. De esta forma se generan los resultados correspondientes a la suma, resta $A - B$, resta inversa $B - A$, desplazamiento lógico hacia la izquierda y desplazamiento lógico hacia la derecha.

La operación que debe utilizarse se determina mediante un código de 3 bits. El selector de operación se encarga de identificar este código y seleccionar la salida del bloque correspondiente. Los códigos utilizados por la calculadora son 001 para la suma, 010 para la resta $A - B$, 011 para la resta $B - A$, 100 para Shift Left y 101 para Shift Right. El código 000 se utiliza para el reinicio, mientras que los códigos 110 y 111 no son utilizados.

Una vez seleccionado el resultado de la operación, este puede ser almacenado en el registro de resultado mediante la señal `ejecutar`. El registro se actualiza en el flanco positivo de la señal de reloj y mantiene el valor almacenado mientras no se ejecute una nueva operación. Además, mediante la señal de reinicio, el contenido del registro puede volver al valor 0000.

El resultado almacenado puede posteriormente ser utilizado como segundo operando de una

nueva operación mediante `sel_op2`, permitiendo realizar una secuencia de cálculos utilizando el resultado obtenido anteriormente.

Todas las operaciones se realizan utilizando únicamente cuatro bits. Por lo tanto, cuando una operación genera un resultado de mayor longitud, los bits que exceden los cuatro bits disponibles son descartados, manteniendo únicamente los cuatro bits menos significativos del resultado.

2.2. Arquitectura

La arquitectura de la calculadora está formada por los bloques de Selector OP2, un sumador de 4 bits, un restador de 4 bits, 2 desplazadores de bits, un selector de operaciones y un registro de resultados. En la siguiente figura se muestra un esquemático de la arquitectura:

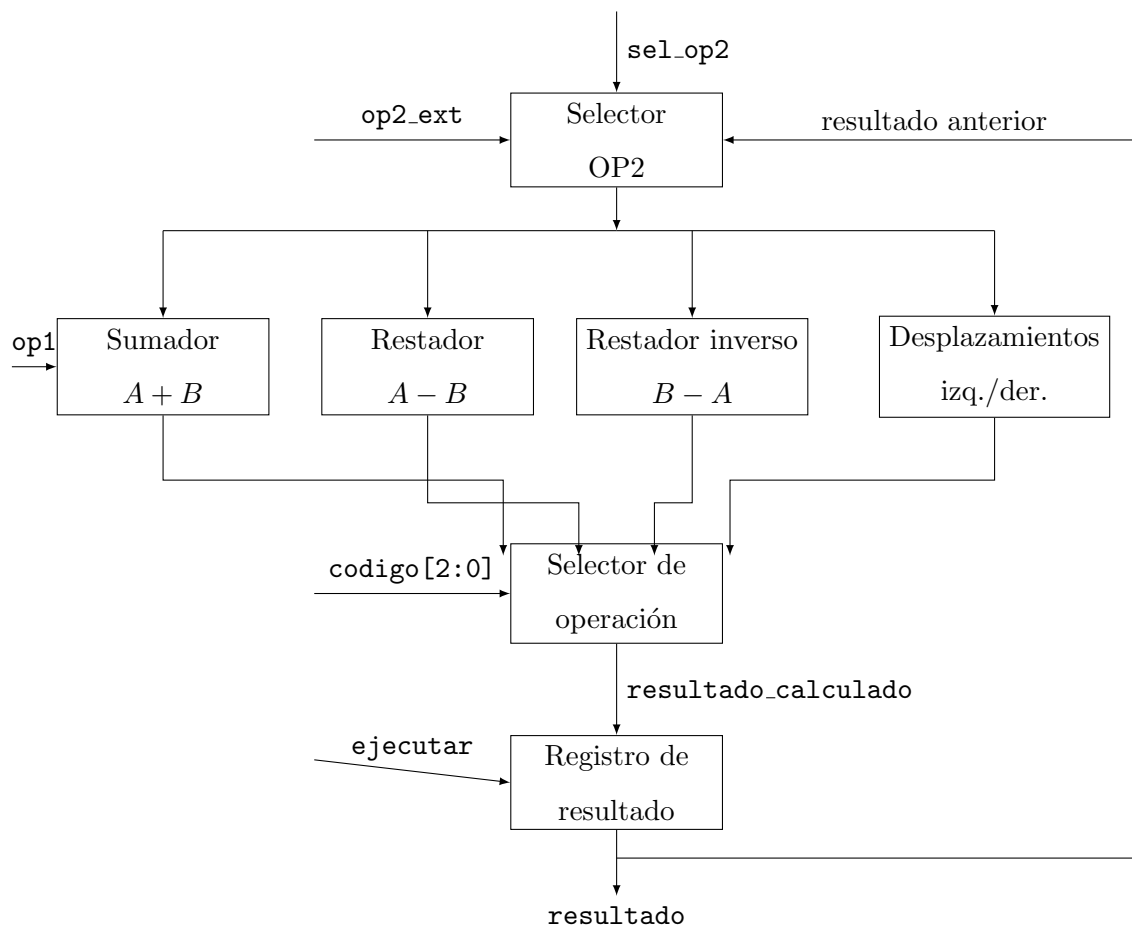


Figura 1: Arquitectura general de la calculadora de 4 bits.

Fuente: Elaboracion Propia

Donde el primer bloque es el selector OP2, el cual decide cual será el segundo operando utilizado, el cual al activar la selección correspondiente, utiliza el resultado que se encuentra almacenado en el registro (el resultado anterior), en otro caso utiliza el valor de op2_ext.

Luego de definir los 2 operandos, se selecciona el bloque operador, el cual puede ser el sumador_4bits para sumar ambos, el restador_4bits para restar el primer operando menos el segundo, el restador inverso de 4 bits para restar el segundo operando menos el primero, y los desplazamientos des_isq y des_der (shift right o shift left) los cuales se encargan de mover un bit a la izquierda o a la derecha según se requiera.

Cada una de las operaciones anteriores genera un resultado independiente, el cual es enviado a selector_operaciones, y este se encarga de mostrar el resultado según corresponda dependiendo del código de 3 bits que se envíe. Los códigos son 000 para el reinicio, 001 para la suma, 010 para la resta, 011 para la resta inversa, 100 para el desplazamiento a la izquierda y 101 para el desplazamiento a la derecha.

Finalmente, el resultado de la operación es enviado al bloque registro_resultado, el cual se encarga de almacenar el resultado para luego enviarlo como resultado, y de ser necesario realizar una retroalimentación para hacer otra operación con el resultado de la anterior.

3. Operaciones de la calculadora

La operación realizada por la calculadora se determina mediante un selector de tres bits. La codificación utilizada corresponde a la indicada en el enunciado del proyecto.

Cuadro 1: Operaciones implementadas por la calculadora.

Código	Operación	Resultado	Descripción
000	Reinicio	$R = 0000$	Establece el resultado almacenado en cero.
001	Suma	$R = A + B$	Suma ambos operandos.
010	Resta	$R = A - B$	Resta el operando B al operando A mediante complemento a dos.
011	Resta inversa	$R = B - A$	Resta el operando A al operando B mediante complemento a dos.
100	Shift Left	$R = A \ll B[1 : 0]$	Desplaza A hacia la izquierda entre cero y tres posiciones.
101	Shift Right	$R = A \gg B[1 : 0]$	Desplaza A hacia la derecha entre cero y tres posiciones.
110	No utilizada	–	Código no utilizado.
111	No utilizada	–	Código no utilizado.

Todas las operaciones se realizan utilizando únicamente cuatro bits. Por lo tanto, cuando una operación genera un resultado de mayor longitud, los bits que exceden los cuatro bits disponibles son descartados, manteniendo únicamente los cuatro bits menos significativos del resultado.

4. Diseño lógico de la calculadora

4.1. Sumador completo de 1 bit

4.1.1. Tabla de verdad

Cuadro 2: Tabla de verdad del sumador completo de 1 bit

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

4.1.2. Mapas de Karnaugh

Cuadro 3: Mapa de Karnaugh para la salida S del sumador completo

$AB \backslash C_{in}$	0	1
00	0	1
01	1	0
11	0	1
10	1	0

Cuadro 4: Mapa de Karnaugh para la salida C_{out} del sumador completo

$AB \backslash C_{in}$	0	1
00	0	0
01	0	1
11	1	1
10	0	1

4.1.3. Expresiones booleanas

A partir de los mapas de Karnaugh se obtienen las siguientes expresiones booleanas para el sumador completo.

Para la salida S :

$$S = \overline{A} \overline{B} C_{in} + \overline{A} B \overline{C_{in}} + A \overline{B} \overline{C_{in}} + A B C_{in} \quad (1)$$

La expresión anterior puede simplificarse utilizando compuertas XOR:

$$S = A \oplus B \oplus C_{in} \quad (2)$$

Para la salida C_{out} :

$$C_{out} = AB + AC_{in} + BC_{in} \quad (3)$$

4.2. Sumador de 4 bits

El sumador de 4 bits se construye mediante cuatro sumadores completos conectados en cascada. Cada etapa realiza la suma de un bit de los operandos A y B , utilizando además el acarreo proveniente de la etapa anterior.

El primer sumador utiliza un acarreo de entrada $C_{in} = 0$. Luego, el acarreo de salida de cada etapa se conecta como acarreo de entrada de la siguiente. De esta forma, se obtiene un resultado de 4 bits. El acarreo final se descarta, ya que la calculadora trabaja únicamente con resultados de 4 bits.

4.3. Restador mediante complemento a 2

La operación de resta se implementó utilizando complemento a dos. Para realizar la resta $A - B$, se invierten los bits del segundo operando B y posteriormente se suma una unidad. De esta forma, la operación puede expresarse como:

$$A - B = A + \overline{B} + 1 \quad (4)$$

Este método permite realizar la resta utilizando los mismos sumadores completos empleados en la operación de suma.

4.3.1. Tabla de verdad

Cuadro 5: Tabla de verdad de la inversión de cada bit de B

B_i	B'_i
0	1
1	0

4.3.2. Mapa de Karnaugh

Cuadro 6: Mapa de Karnaugh para la inversión de B_i

B_i	0	1
B'_i	1	0

4.3.3. Expresiones booleanas

Cada bit del segundo operando se invierte mediante:

$$B'_i = \overline{B_i} \quad (5)$$

Posteriormente, cada etapa utiliza las expresiones del sumador completo:

$$S_i = A_i \oplus B'_i \oplus C_i \quad (6)$$

$$C_{i+1} = A_i B'_i + A_i C_i + B'_i C_i \quad (7)$$

Para el bit menos significativo se establece:

$$C_0 = 1 \quad (8)$$

De esta manera se obtiene:

$$A + \overline{B} + 1 = A - B \quad (9)$$

4.4. Multiplexor 2:1

4.4.1. Tabla de verdad

Cuadro 7: Tabla de verdad del multiplexor 2:1

SEL	I_0	I_1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

4.4.2. Mapas de Karnaugh

Cuadro 8: Mapa de Karnaugh del multiplexor 2:1

$SEL \backslash I_0 I_1$	00	01	11	10
0	0	0	1	1
1	0	1	1	0

4.4.3. Expresiones booleanas

A partir del mapa de Karnaugh del multiplexor 2:1 se obtiene:

$$Y = \overline{SEL}I_0 + SELI_1 \quad (10)$$

Esta expresión indica que cuando $SEL = 0$ se selecciona I_0 , mientras que cuando $SEL = 1$ se selecciona I_1 .

4.5. Selector de operación

El selector de operación determina cuál de los resultados generados por los distintos bloques de la calculadora será seleccionado de acuerdo con el código de operación $S_2S_1S_0$. Cada combinación válida corresponde a una operación específica de la calculadora.

4.5.1. Tabla de verdad

Cuadro 9: Tabla de verdad del selector de operación

S_2	S_1	S_0	Operación
0	0	0	Reinicio
0	0	1	Suma
0	1	0	Resta $A - B$
0	1	1	Resta $B - A$
1	0	0	Shift Left
1	0	1	Shift Right
1	1	0	No utilizada
1	1	1	No utilizada

4.5.2. Mapas de Karnaugh

Para obtener las señales de selección de cada operación se utilizan los tres bits del código de operación $S_2S_1S_0$.

Cuadro 10: Mapa de Karnaugh para la operación de reinicio

$S_2 \backslash S_1S_0$	00	01	11	10
0	1	0	0	0
1	0	0	0	0

Cuadro 11: Mapa de Karnaugh para la operación de suma

$S_2 \backslash S_1 S_0$	00	01	11	10
0	0	1	0	0
1	0	0	0	0

Cuadro 12: Mapa de Karnaugh para la operación de resta $A - B$

$S_2 \backslash S_1 S_0$	00	01	11	10
0	0	0	1	0
1	0	0	0	0

Cuadro 13: Mapa de Karnaugh para la operación de resta $B - A$

$S_2 \backslash S_1 S_0$	00	01	11	10
0	0	0	0	1
1	0	0	0	0

Cuadro 14: Mapa de Karnaugh para la operación Shift Left

$S_2 \backslash S_1 S_0$	00	01	11	10
0	0	0	0	0
1	0	0	0	1

Cuadro 15: Mapa de Karnaugh para la operación Shift Right

$S_2 \backslash S_1 S_0$	00	01	11	10
0	0	0	0	0
1	0	1	0	0

4.5.3. Expresiones booleanas

A partir de los mapas de Karnaugh se obtienen las señales que habilitan cada una de las operaciones de la calculadora:

$$SEL_{reset} = \overline{S_2} \overline{S_1} \overline{S_0} \quad (11)$$

$$SEL_{suma} = \overline{S_2} \overline{S_1} S_0 \quad (12)$$

$$SEL_{restaAB} = \overline{S_2} S_1 \overline{S_0} \quad (13)$$

$$SEL_{restaBA} = \overline{S_2} S_1 S_0 \quad (14)$$

$$SEL_{izq} = S_2 \overline{S_1} \overline{S_0} \quad (15)$$

$$SEL_{der} = S_2 \overline{S_1} S_0 \quad (16)$$

Cada señal toma el valor lógico 1 únicamente cuando el código de operación corresponde a la operación asociada. Estas señales permiten seleccionar el resultado que será enviado a la salida de la calculadora.

4.6. Shift Left

En esta sección, A_3 , A_2 , A_1 y A_0 representan los cuatro bits del operando A , donde A_3 corresponde al bit más significativo y A_0 al menos significativo. Los bits C_1C_0 , correspondientes a $op2[1 : 0]$, determinan la cantidad de posiciones que se desplaza el operando hacia la izquierda, pudiendo realizar desplazamientos de 0 a 3 posiciones.

4.6.1. Tabla de verdad

Cuadro 16: Tabla de funcionamiento del desplazamiento a la izquierda

C_1	C_0	R_3	R_2	R_1	R_0
0	0	A_3	A_2	A_1	A_0
0	1	A_2	A_1	A_0	0
1	0	A_1	A_0	0	0
1	1	A_0	0	0	0

4.6.2. Mapas de Karnaugh

Cuadro 17: Mapa de selección para R_3 del Shift Left

$C_1 \backslash C_0$	0	1
0	A_3	A_2
1	A_1	A_0

Cuadro 18: Mapa de selección para R_2 del Shift Left

$C_1 \backslash C_0$	0	1
0	A_2	A_1
1	A_0	0

Cuadro 19: Mapa de selección para R_1 del Shift Left

$C_1 \backslash C_0$	0	1
0	A_1	A_0
1	0	0

Cuadro 20: Mapa de selección para R_0 del Shift Left

$C_1 \backslash C_0$	0	1
0	A_0	0
1	0	0

4.6.3. Expresiones booleanas

Para el desplazamiento hacia la izquierda, la cantidad de posiciones está determinada por los bits C_1C_0 , correspondientes a $op2[1 : 0]$. Las expresiones booleanas obtenidas son:

$$R_3 = \bar{C}_1\bar{C}_0A_3 + \bar{C}_1C_0A_2 + C_1\bar{C}_0A_1 + C_1C_0A_0 \quad (17)$$

$$R_2 = \bar{C}_1\bar{C}_0A_2 + \bar{C}_1C_0A_1 + C_1\bar{C}_0A_0 \quad (18)$$

$$R_1 = \bar{C}_1\bar{C}_0A_1 + \bar{C}_1C_0A_0 \quad (19)$$

$$R_0 = \bar{C}_1\bar{C}_0A_0 \quad (20)$$

4.7. Shift Right

En esta sección, A_3 , A_2 , A_1 y A_0 representan los cuatro bits del operando A , donde A_3 corresponde al bit más significativo y A_0 al menos significativo. Los bits C_1C_0 , correspondientes a $op2[1:0]$, determinan la cantidad de posiciones que se desplaza el operando hacia la derecha, pudiendo realizar desplazamientos de 0 a 3 posiciones.

4.7.1. Tabla de verdad

Cuadro 21: Tabla de funcionamiento del desplazamiento a la derecha

C_1	C_0	R_3	R_2	R_1	R_0
0	0	A_3	A_2	A_1	A_0
0	1	0	A_3	A_2	A_1
1	0	0	0	A_3	A_2
1	1	0	0	0	A_3

4.7.2. Mapas de Karnaugh

Cuadro 22: Mapa de selección para R_3 del Shift Right

$C_1 \setminus C_0$	0	1
0	A_3	0
1	0	0

Cuadro 23: Mapa de selección para R_2 del Shift Right

$C_1 \setminus C_0$	0	1
0	A_2	A_3
1	0	0

Cuadro 24: Mapa de selección para R_1 del Shift Right

$C_1 \backslash C_0$	0	1
0	A_1	A_2
1	A_3	0

Cuadro 25: Mapa de selección para R_0 del Shift Right

$C_1 \backslash C_0$	0	1
0	A_0	A_1
1	A_2	A_3

4.7.3. Expresiones booleanas

Para el desplazamiento hacia la derecha, la cantidad de posiciones está determinada por los bits C_1C_0 , correspondientes a $op2[1 : 0]$. Las expresiones booleanas obtenidas son:

$$R_3 = \bar{C}_1 \bar{C}_0 A_3 \quad (21)$$

$$R_2 = \bar{C}_1 \bar{C}_0 A_2 + \bar{C}_1 C_0 A_3 \quad (22)$$

$$R_1 = \bar{C}_1 \bar{C}_0 A_1 + \bar{C}_1 C_0 A_2 + C_1 \bar{C}_0 A_3 \quad (23)$$

$$R_0 = \bar{C}_1 \bar{C}_0 A_0 + \bar{C}_1 C_0 A_1 + C_1 \bar{C}_0 A_2 + C_1 C_0 A_3 \quad (24)$$

4.8. Registro y señal de ejecución

El resultado de la operación se almacena en un registro de 4 bits. Este registro permite conservar el resultado obtenido hasta que se ejecute una nueva operación o se active la señal de reinicio.

4.8.1. Tabla de funcionamiento

Cuadro 26: Tabla de funcionamiento del registro de resultado

$RESET$	$EXEC$	Q_{nuevo}
0	0	Q_{anterior}
0	1	D
1	0	0000
1	1	0000

4.8.2. Funcionamiento

El registro se actualiza en cada flanco positivo de la señal de reloj. La señal $RESET$ tiene prioridad sobre la señal $EXEC$.

Cuando $RESET = 1$, el contenido del registro se reinicia y el resultado almacenado pasa a ser 0000.

Cuando $RESET = 0$ y $EXEC = 1$, el registro almacena el nuevo resultado calculado, representado por la entrada D .

Finalmente, cuando $RESET = 0$ y $EXEC = 0$, el registro no modifica su contenido y mantiene el resultado de la operación anterior.

4.8.3. Implementación

La implementación se realiza mediante lógica secuencial sensible al flanco positivo del reloj. El registro posee una entrada de datos de 4 bits, correspondiente al resultado generado por el selector de operación.

La señal $EXEC$ actúa como una señal de habilitación de escritura: cuando se encuentra activa, el nuevo resultado es almacenado en el registro. En caso contrario, el valor anterior se conserva.

La señal *RESET* permite borrar el contenido almacenado y tiene prioridad sobre la señal *EXEC*. Por lo tanto, si ambas señales se encuentran activas simultáneamente, el registro almacena 0000.

5. Implementación mediante compuertas

A partir de las expresiones booleanas obtenidas en la etapa de diseño lógico, se realizó la implementación de las distintas operaciones de la calculadora mediante compuertas lógicas. Para la lógica combinacional se utilizaron primitivas de Verilog, tales como `and`, `or`, `not`, `xor` y `buf`, evitando utilizar operadores aritméticos o de desplazamiento para implementar directamente las operaciones.

La calculadora fue dividida en distintos archivos y módulos, donde cada uno cumple una función específica. Posteriormente, estos módulos se conectan para obtener el resultado final de la operación seleccionada.

5.1. Implementación del sumador

La implementación del sumador se encuentra principalmente en los archivos `sumador_4bits.v` y en el módulo correspondiente al sumador completo de un bit.

El sumador completo de un bit se implementó utilizando compuertas XOR, AND y OR. La salida de suma se obtiene mediante compuertas XOR, mientras que el acarreo se genera a partir de compuertas AND y OR.

En el archivo `sumador_4bits.v` se construye el sumador de 4 bits conectando cuatro sumadores completos en cascada. El acarreo de salida de cada etapa se utiliza como acarreo de entrada de la siguiente, comenzando con un acarreo de entrada igual a cero.

5.2. Implementación del restador

La implementación de la resta se encuentra en el archivo `restador_4bits.v`.

La resta se realizó mediante complemento a dos, reutilizando la estructura del sumador completo. Para realizar $A - B$, cada bit del operando B se invierte mediante compuertas NOT y el acarreo de entrada del primer sumador se establece en uno.

De esta manera, el circuito realiza la operación

$$A + \overline{B} + 1 = A - B. \quad (25)$$

La operación $B - A$ utiliza el mismo módulo de resta, pero intercambiando el orden de los operandos al realizar la conexión dentro del módulo principal de la calculadora.

5.3. Implementación del Shift Left

El desplazamiento lógico hacia la izquierda se implementó en el archivo `des_izq.v`.

Este módulo permite desplazar el operando *A* entre 0 y 3 posiciones, dependiendo del valor de los bits `op2[1 : 0]`. La selección de los bits correspondientes se realiza mediante primitivas de compuertas, utilizando principalmente `not`, `and`, `or` y `buf`.

Las posiciones que quedan disponibles producto del desplazamiento se completan con ceros. De esta forma, el desplazamiento se implementa sin utilizar directamente el operador « de Verilog.

»5.4. Implementación del Shift Right

»El desplazamiento lógico hacia la derecha se implementó en el archivo `des_der.v`.

»Al igual que en el desplazamiento hacia la izquierda, los bits `op2[1 : 0]` determinan si el operando *A* se desplaza 0, 1, 2 o 3 posiciones. La selección de los bits de salida se realiza mediante compuertas lógicas y las posiciones que quedan disponibles se completan con ceros.

»De esta manera, el desplazamiento se implementa mediante primitivas lógicas sin utilizar directamente el operador » de Verilog.

5.5. Implementación del selector del segundo operando

La selección del segundo operando se implementó en el archivo `selector_ope2.v`.

Este módulo funciona como un multiplexor que permite seleccionar entre el segundo operando ingresado externamente y el resultado almacenado de la operación anterior. La selección se realiza mediante una señal de control y utilizando compuertas lógicas.

Cuando la señal de selección se encuentra en cero se utiliza el operando externo, mientras que cuando se encuentra en uno se utiliza el resultado de la operación anterior.

5.6. Implementación del selector de operación

La selección del resultado correspondiente a cada operación se implementó en el archivo `selector_operaciones.v`.

Este módulo recibe el código de operación de tres bits. Las distintas combinaciones se detectan utilizando compuertas NOT y AND, generando las señales necesarias para habilitar el resultado

correspondiente a suma, resta $A - B$, resta $B - A$, Shift Left o Shift Right.

Posteriormente, los resultados habilitados se combinan mediante compuertas OR para obtener una única salida correspondiente a la operación seleccionada.

5.7. Implementación del registro de resultado

El almacenamiento del resultado se implementó en el archivo `registro_resultado.v`.

A diferencia de los módulos anteriores, este bloque corresponde a lógica secuencial. El registro se actualiza en el flanco positivo de la señal de reloj. Si la señal de reinicio se encuentra activa, el resultado se establece en cero. En caso contrario, cuando la señal de ejecución está activa, se almacena el nuevo resultado calculado.

Si la señal de ejecución no se encuentra activa, el registro mantiene el resultado de la operación anterior.

5.8. Integración de los módulos

Finalmente, la conexión entre los distintos bloques de la calculadora se realiza en el archivo `calcul_4bits.v`.

En este módulo se conectan el selector del segundo operando, el sumador, los restadores, los bloques de desplazamiento, el selector de operación y el registro de resultado. De esta manera, cada bloque realiza una función específica y el módulo principal se encarga de interconectarlos para obtener y almacenar el resultado final.

6. Declaración de uso de IA

En el presente informe se utilizó IA para ayudar con la redaccion y la correccion de la ortografía, junto con la distribución y el orden del mismo.

7. Bibliografía

OpenStax. University Physics Volume 2. Cap. 13, sec. 13.5, “Eddy Currents”, pp. 602–606